Analyse des logs Firewall Iptables

Code

Projet SISE-OPSIE 2026 — Revue d'utilisation des règles

AUTEUR·RICE
Équipe SISE-OPSIE

DATE DE PUBLICATION
02 mars 2026

Introduction

Ce rapport présente une analyse exploratoire des logs du Firewall Iptables (FW=6) couvrant la période **novembre 2025 à février 2026**. Les données proviennent du fichier `logs_export.parquet` et ont été filtrées et nettoyées avant analyse.

L'objectif est de fournir une revue d'utilisation des règles de filtrage, conformément aux exigences normatives (ISO 27001, ANSSI).

---

1. Chargement et préparation des données

1.1 Import des librairies

▼ Code

```
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
import plotly.graph_objects as go
from IPython.display import display
import re
```

1.2 Chargement du fichier Parquet

▼ Code

```
pattern = re.compile(

    r'^(\w{3}\s+\d+\s+\d+:\d+:\d+)\s+'
    r'\S+\s+kernel:.*?\s+'
    r'(PERMIT|DENY)\s+'
    r'FW=\S+\s+'
    r'RULE=(\S+)\s+'
    r'IN=(\S*)\s+'
```

```
    r'OUT=(\S*)\s+'
    r'.*?'
    r'SRC=(\S+)\s+'
    r'DST=(\S+)\s+'
    r'.*?'
    r'PROTO=(\S+)\s+'
    r'SPT=(\d+)\s+'
    r'DPT=(\d+)'
)

def parse_line(line):
    m = pattern.search(line)
    if m:
        return m.groups()
    return None

# test sur une ligne

sample = 'Mar 20 01:29:24 159.84.146.99 kernel: [54783294.108218] DENY FW=6 RULE=999 IN=eth0 OU

print(parse_line(sample))
```

```
('Mar 20 01:29:24', 'DENY', '999', 'eth0', '', '94.102.61.47', '159.84.146.99', 'TCP',
'52502', '3178')
```

▼ Code

```
cols = ["datetime", "action", "rule_id", "interface_in", "interface_out",
"ip_src", "ip_dst", "protocol", "port_src", "port_dst"]

with open("log_brute.log", "r") as f:
    rows = [parse_line(line) for line in f]

df = pd.DataFrame([r for r in rows if r is not None], columns=cols)

print(df.shape)

print(df.head(3))
```

```
(11997646, 10)
          datetime  action rule_id interface_in interface_out          ip_src  \
0  Mar 20 01:29:24    DENY     999         eth0                  94.102.61.47
1  Mar 20 01:29:25    DENY     999         eth0                176.111.174.85
2  Mar 20 01:29:27  PERMIT       1         eth0                 66.249.65.106

          ip_dst protocol port_src port_dst
0  159.84.146.99      TCP    52502     3178
1  159.84.146.99      TCP    48739     2231
2  159.84.146.99      TCP    50501      443
```

1.3 Typage des colonnes

▼ Code

```
df["datetime"] = pd.to_datetime(df["datetime"], format="%b %d %H:%M:%S")
df["datetime"] = df["datetime"].apply(
    lambda x: x.replace(year=2026) if x.month<= 2 else x.replace(year=2025)
)
df["port_src"]     = pd.to_numeric(df["port_src"],errors="coerce")
df["port_dst"]     = pd.to_numeric(df["port_dst"],errors="coerce")
df["rule_id"]      = pd.to_numeric(df["rule_id"], errors="coerce")

df_filtered = df[(df["datetime"] >= "2025-11-01") & (df["datetime"] < "2026-03-01")]

print(df_filtered.shape)

print(df_filtered["datetime"].dt.to_period("M").value_counts().sort_index())
```

```
(4572903, 10)
datetime
2025-11    1212335
2025-12    1620630
2026-01    1250863
2026-02     489075
Freq: M, Name: count, dtype: int64
```

1.4 Filtrage temporel et nettoyage

▼ Code

```
# Filtrage sur la période nov 2025 – fév 2026
df = df[(df["datetime"] >= "2025-11-01") & (df["datetime"] < "2026-03-01")]

print(f"\nDimensions du dataframe final : {df.shape[0]:,} lignes × {df.shape[1]} colonnes")
print(df.dtypes)
```

```
Dimensions du dataframe final : 4,572,903 lignes × 10 colonnes
datetime         datetime64[ns]
action                   object
rule_id                   int64
interface_in             object
interface_out            object
ip_src                   object
ip_dst                   object
protocol                 object
```

```
protocol                  object
port_src                   int64
port_dst                   int64
dtype: object
```

1.5 Aperçu des données

▼ Code

```
display(df.head(10).style.set_caption("Aperçu des 10 premières lignes"))
```

Table 1: Aperçu des 10 premières lignes

	datetime	action	rule_id	interface_in	interface_out	ip_src	ip_dst	protocol	port_src	port_
7424743	2025-11-12 09:56:37	DENY	999	eth0		77.90.185.64	159.84.146.99	TCP	55366	651
7424744	2025-11-12 09:56:37	PERMIT	1	eth0		47.128.20.252	159.84.146.99	TCP	14436	44
7424745	2025-11-12 09:56:38	PERMIT	1	eth0		23.22.35.162	159.84.146.99	TCP	55973	44
7424746	2025-11-12 09:56:39	DENY	999	eth0		79.124.60.150	159.84.146.99	TCP	53744	562
7424747	2025-11-12 09:56:39	PERMIT	1	eth0		47.128.20.252	159.84.146.99	TCP	25404	44
7424748	2025-11-12 09:56:39	PERMIT	1	eth0		47.128.20.252	159.84.146.99	TCP	25408	44
7424749	2025-11-12 09:56:39	PERMIT	1	eth0		47.128.20.252	159.84.146.99	TCP	25442	44
7424750	2025-11-12 09:56:39	PERMIT	1	eth0		47.128.20.252	159.84.146.99	TCP	25424	44
7424751	2025-11-12 09:56:39	PERMIT	1	eth0		47.128.20.252	159.84.146.99	TCP	25500	44
7424752	2025-11-12 09:56:40	PERMIT	1	eth0		76.33.9.128	159.84.146.99	TCP	43172	44

2. Déduction des protocoles

Dans les logs, certains protocoles peuvent être absents ou mal nommés. La déduction est effectuée à partir des ports de destination connus.

▼ Code

```
# Ports UDP connus (RFC standards)
UDP_PORTS = {53, 67, 68, 69, 123, 161, 162, 514, 520, 1900, 5353}

def deduce_protocol(row):
    proto = str(row["protocol"]).strip().upper()
    if proto == "TCP":
        return "TCP"
    elif proto == "UDP":
        return "UDP"
    elif proto in ["", "NA", "N.A", "NONE", "NAN"]:
        return "UDP" if row["port_dst"] in UDP_PORTS else "TCP"
    else:
        return "OTHER"

df["protocol_clean"] = df.apply(deduce_protocol, axis=1)

print("Distribution des protocoles après déduction :")
print(df["protocol_clean"].value_counts())
```

```
Distribution des protocoles après déduction :
protocol_clean
TCP    4572903
Name: count, dtype: int64
```

---

3. Classement des règles les plus utilisées

▼ Code

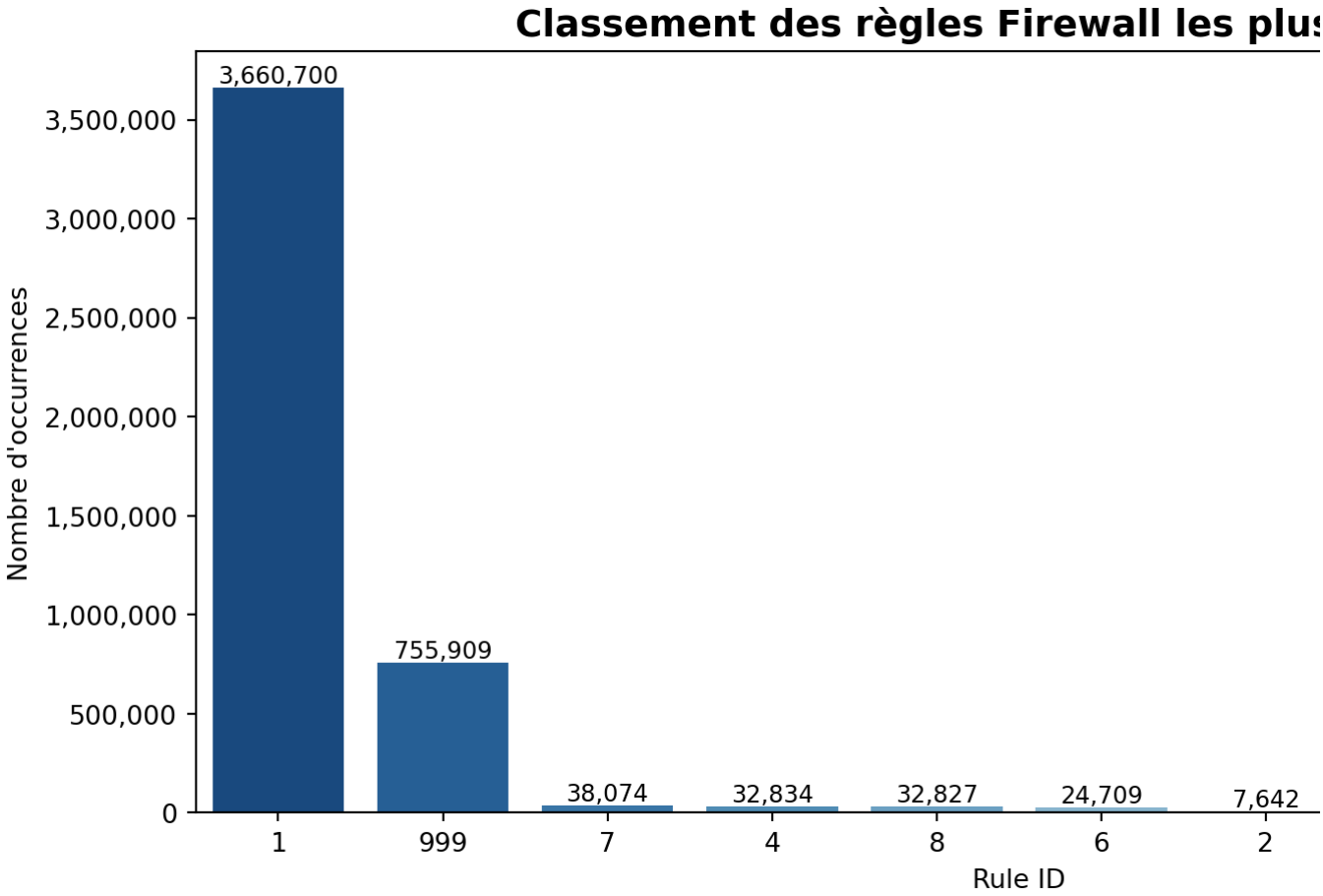
```
rule_counts = (
    df.groupby("rule_id")
      .size()
      .reset_index(name="count")
      .sort_values("count", ascending=False)
)

display(
    rule_counts.style
               .bar(subset=["count"], color="#4C72B0")
               .format({"count": "{:,}", "rule_id": "{:.0f}"})
               .set_caption("Classement des règles par nombre d'occurrences")
)
```

Table 2: Classement des règles par nombre d'occurrences

	rule_id	count
0	1	3,660,700

9	999	755,909
5	7	38,074
2	4	32,834
6	8	32,827
4	6	24,709
1	2	7,642
7	9	7,642
3	5	6,656
8	11	5,910

▸ Code

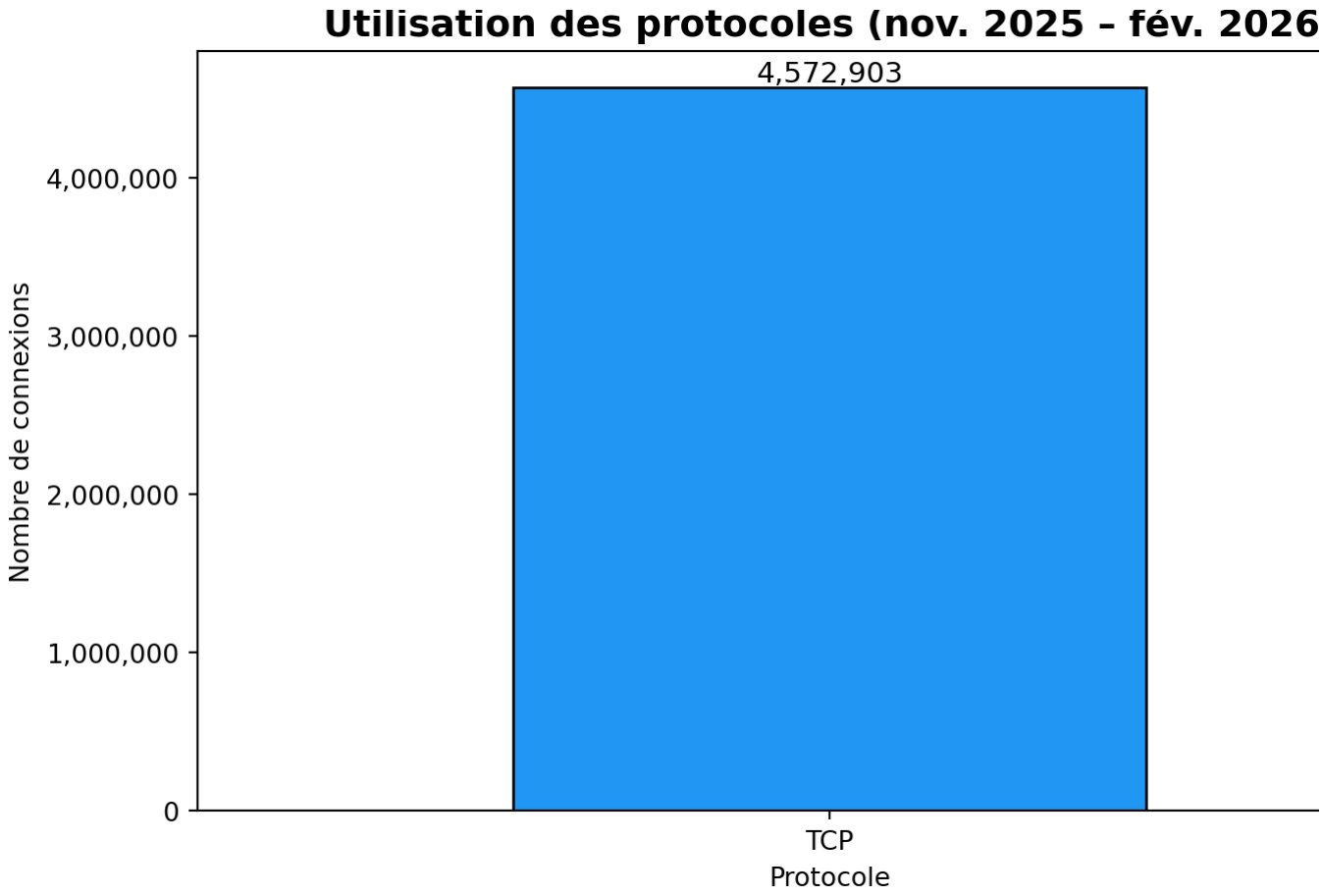


> **Note :** La règle **999** correspond à la règle *cleanup* (règle par défaut qui capture tout le trafic non matché).

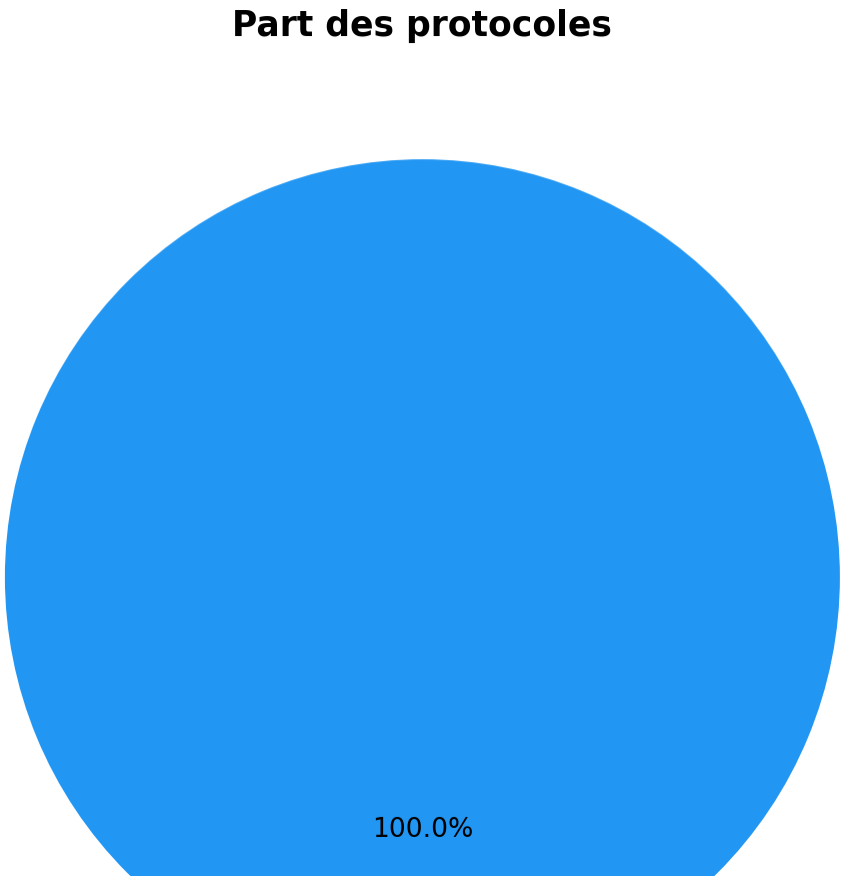
---

4. Répartition des protocoles

▸ Code


Utilisation des protocoles (nov. 2025 – fév. 2026
4,572,903
4,000,000
3,000,000
2,000,000
1,000,000
0
Nombre de connexions
TCP
Protocole

▶ Code


Part des protocoles
100.0%

TCP

---

5. Top 10 des règles les plus utilisées — Protocole UDP

▼ Code

```
top10_udp = (
    df[df["protocol_clean"] == "UDP"]
      .groupby("rule_id")
      .size()
      .reset_index(name="count")
      .sort_values("count", ascending=False)
      .head(10)
)

display(
    top10_udp.style
             .bar(subset=["count"], color="#FF9800")
             .format({"count": "{:,}", "rule_id": "{:.0f}"})
             .set_caption("Top 10 des règles — Protocole UDP")
)
```

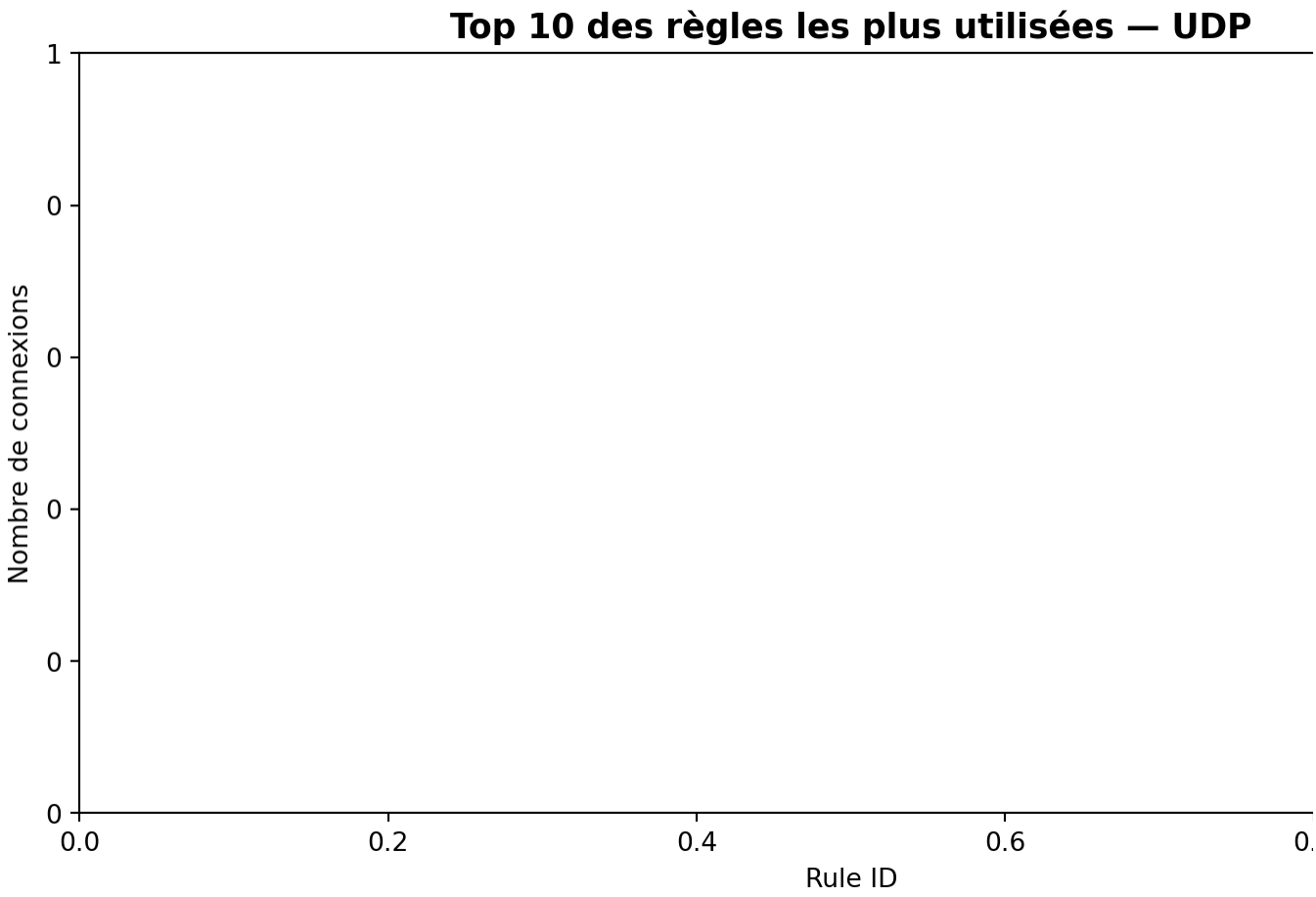
Table 3: Top 10 des règles — Protocole UDP

rule_id	count

▼ Code

```
fig, ax = plt.subplots(figsize=(9, 5))
sns.barplot(
    data=top10_udp,
    x=top10_udp["rule_id"].astype(int).astype(str),
    y="count",
    palette="Oranges_r",
    ax=ax
)
ax.set_title("Top 10 des règles les plus utilisées — UDP", fontsize=13, fontweight="bold")
ax.set_xlabel("Rule ID")
ax.set_ylabel("Nombre de connexions")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{int(x):,}"))
for p in ax.patches:
```

```
    ax.annotate(f"{int(p.get_height()):,}",
                (p.get_x() + p.get_width() / 2, p.get_height()),
                ha="center", va="bottom", fontsize=9)
plt.tight_layout()
plt.show()
```



---

6. Top 5 des règles les plus utilisées — Protocole TCP

▼ Code

```
top5_tcp = (
    df[df["protocol_clean"] == "TCP"]
      .groupby("rule_id")
      .size()
      .reset_index(name="count")
      .sort_values("count", ascending=False)
      .head(5)
)

display(
    top5_tcp.style
```

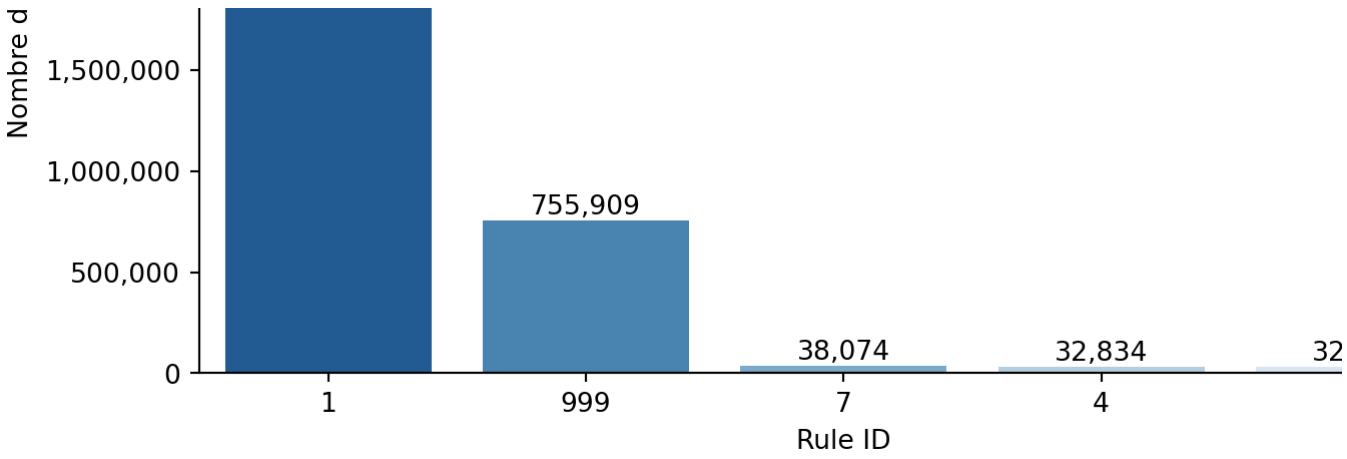
```
            .bar(subset=["count"], color="#2196F3")
            .format({"count": "{:,}", "rule_id": "{:.0f}"})
            .set_caption("Top 5 des règles — Protocole TCP")
)
```

Table 4: Top 5 des règles — Protocole TCP

	rule_id	count
0	1	3,660,700
9	999	755,909
5	7	38,074
2	4	32,834
6	8	32,827

▼ Code

```
fig, ax = plt.subplots(figsize=(8, 5))
sns.barplot(
    data=top5_tcp,
    x=top5_tcp["rule_id"].astype(int).astype(str),
    y="count",
    palette="Blues_r",
    ax=ax
)
ax.set_title("Top 5 des règles les plus utilisées — TCP", fontsize=13, fontweight="bold")
ax.set_xlabel("Rule ID")
ax.set_ylabel("Nombre de connexions")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"{int(x):,}"))
for p in ax.patches:
    ax.annotate(f"{int(p.get_height()):,}",
                (p.get_x() + p.get_width() / 2, p.get_height()),
                ha="center", va="bottom", fontsize=10)
plt.tight_layout()
plt.show()
```



---

7. Rapprochement Règles / Ports de destination / Action (TCP)

Cette analyse croise les règles firewall avec les ports de destination et les actions associées, **uniquement pour le protocole TCP**.

7.1 Tableau de rapprochement

▼ Code

```
tcp_df = df[df["protocol_clean"] == "TCP"]

rule_port_action = (
    tcp_df.groupby(["rule_id", "port_dst", "action"])
          .size()
          .reset_index(name="count")
          .sort_values(["rule_id", "count"], ascending=[True, False])
)

# Affichage des 30 premières lignes
display(
    rule_port_action.head(30).style
        .format({"count": "{:,}", "rule_id": "{:.0f}", "port_dst": "{:.0f}"})
        .applymap(lambda v: "background-color: #c8e6c9" if v == "PERMIT"
                  else ("background-color: #ffcdd2" if v == "DENY" else ""),
                  subset=["action"])
        .set_caption("Rapprochement Règles / Ports / Actions — TCP (30 premières lignes)")
)
```

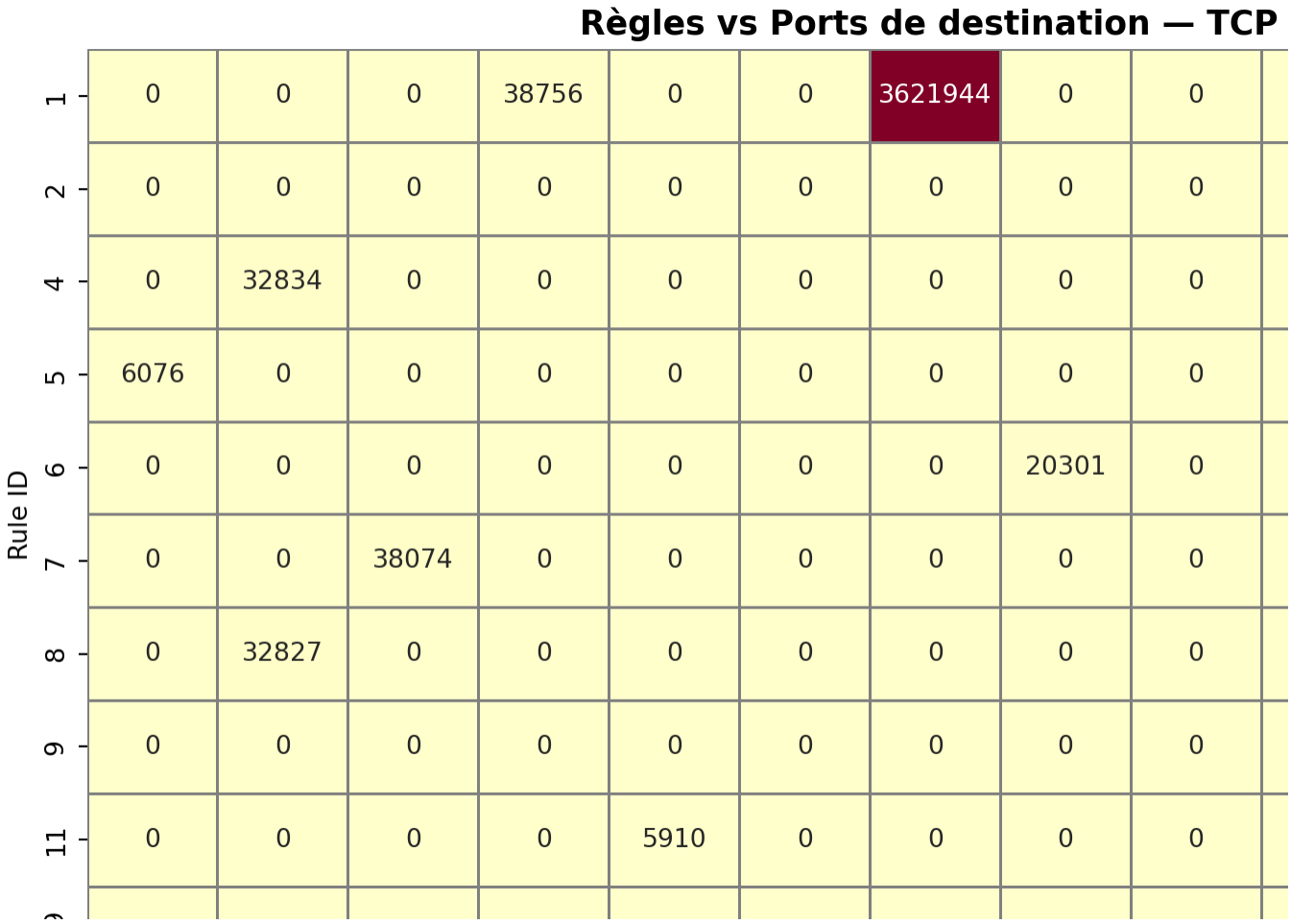
Table 5: Rapprochement Règles / Ports / Actions — TCP (30 premières lignes)

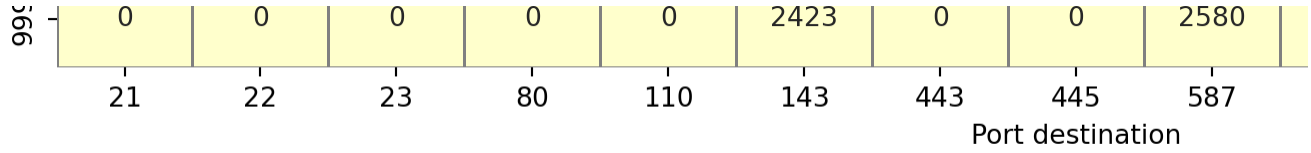
	rule_id	port_dst	action	count
1	1	443	PERMIT	3,621,944
0	1	80	PERMIT	38,756
2	2	3306	PERMIT	7,642
3	4	22	PERMIT	32,834
5	5	21	DENY	6,076
4	5	20	DENY	580
7	6	445	DENY	20,301
8	6	3389	DENY	4,058
6	6	135	DENY	350
9	7	23	DENY	38,074
10	8	22	PERMIT	32,827
11	9	3306	PERMIT	7,642
12	11	110	PERMIT	5,910
8079	999	8080	DENY	7,129
8727	999	8728	DENY	3,118
589	999	587	DENY	2,580
2376	999	2375	DENY	2,555
148	999	143	DENY	2,423
1435	999	1433	DENY	2,320
8442	999	8443	DENY	2,221
5554	999	5555	DENY	2,192
467	999	465	DENY	2,150
997	999	995	DENY	1,843
995	999	993	DENY	1,834
2089	999	2087	DENY	1,699
2224	999	2222	DENY	1,692
2324	999	2323	DENY	1,453
88	999	81	DENY	1,352
8087	999	8088	DENY	1,350
27016	999	27017	DENY	1,230

7.2 Heatmap Règles × Ports de destination (Top 15 ports)

▼ Code

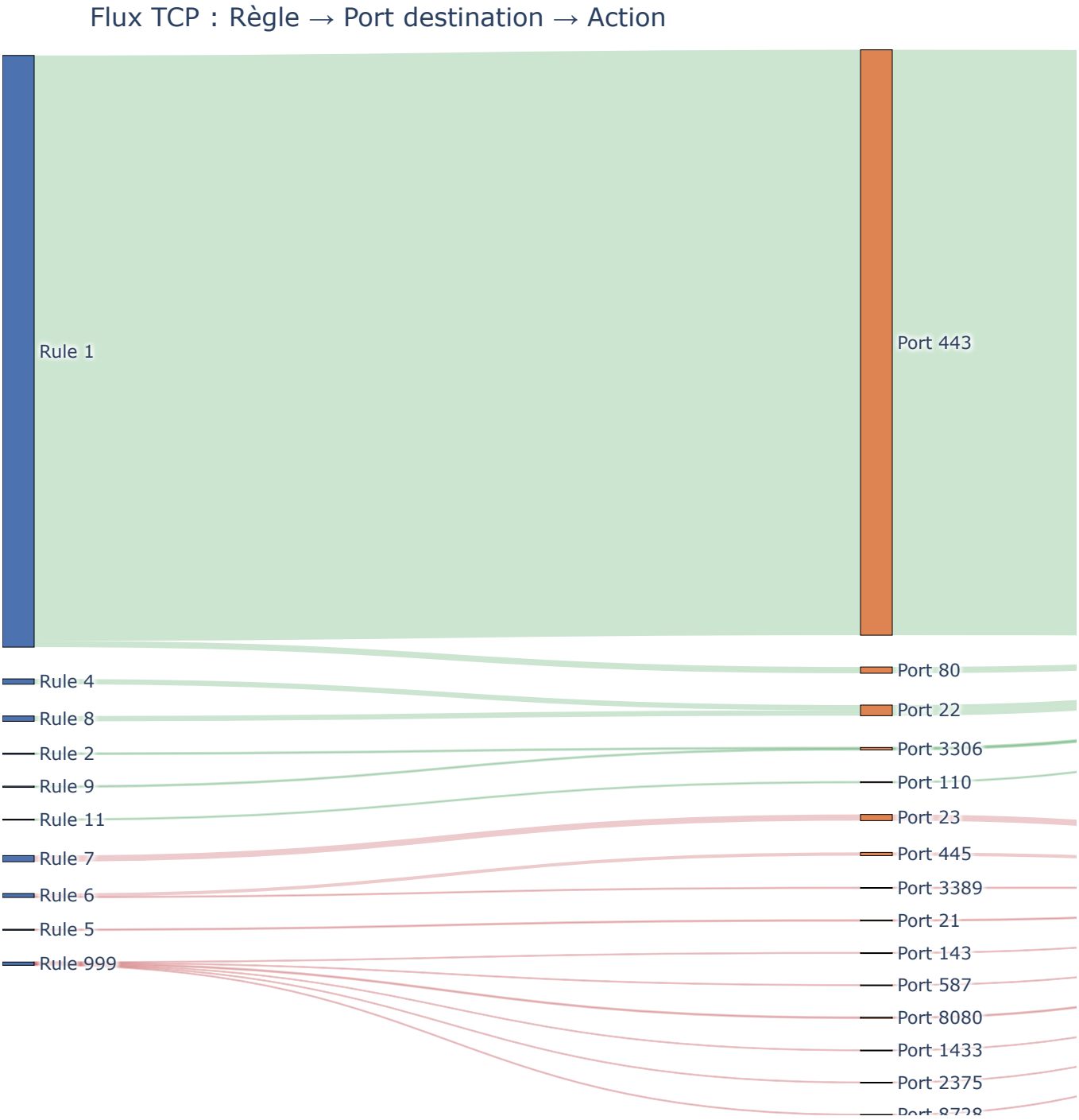
```
top_ports = tcp_df["port_dst"].value_counts().head(15).index
heatmap_data = (
    tcp_df[tcp_df["port_dst"].isin(top_ports)]
          .groupby(["rule_id", "port_dst"])
          .size()
          .unstack(fill_value=0)
)

fig, ax = plt.subplots(figsize=(14, 6))
sns.heatmap(
    heatmap_data,
    annot=True,
    fmt="d",
    cmap="YlOrRd",
    ax=ax,
    linewidths=0.5,
    linecolor="grey"
)
ax.set_title("Règles vs Ports de destination — TCP (Top 15 ports)", fontsize=13, fontweight="bo
ax.set_xlabel("Port destination")
ax.set_ylabel("Rule ID")
plt.tight_layout()
plt.show()
```





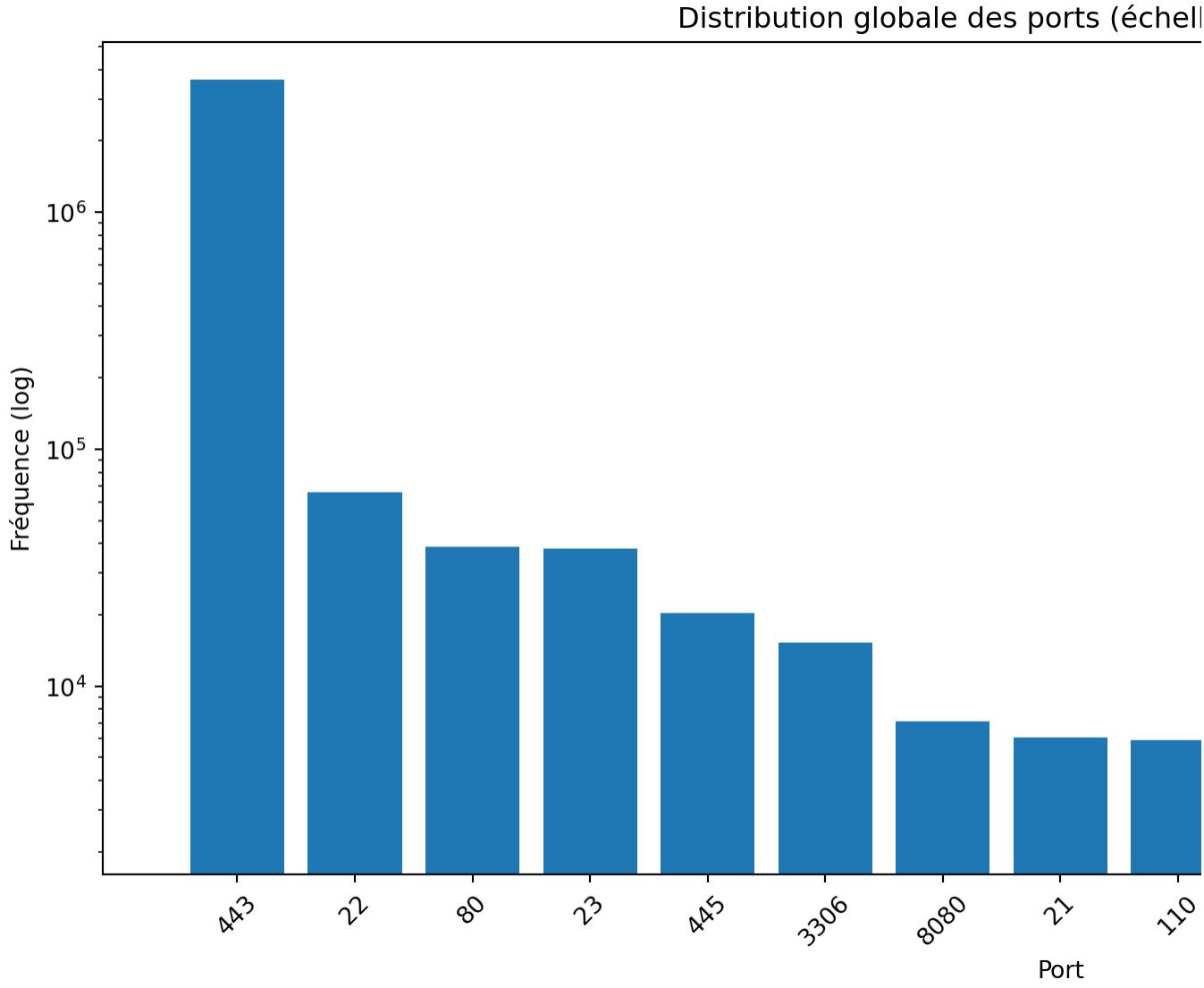
7.3 Diagramme de Sankey — Flux TCP : Règle → Port → Action

▸ Code



8. Tops ports

▶ Code



Rapport généré automatiquement via Quarto — Projet SISE-OPSIE 2026